Transform & Duplex Streams

10 min read Updated 24 Jul 2026 Topic 28 of 70

Bookmark Mark done

Why it matters

Readable and Writable move data. Transform and Duplex are how you *do work to data while it moves* — without ever holding the whole thing in memory. Every time you gzip a response, encrypt a file, parse a multi-gigabyte CSV, or hash an upload as it arrives, you are using a Transform stream. They are the workhorses of real Node pipelines, and they are what let you build a processing chain — read → decompress → parse → filter → serialize → write — that runs in constant memory over an input larger than RAM.

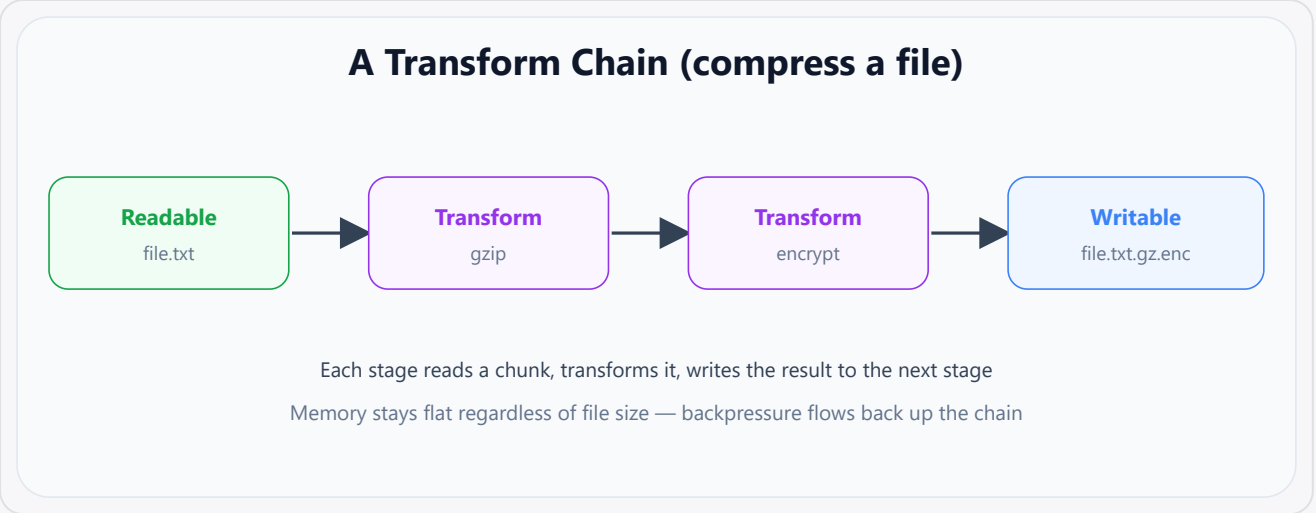
Interviewers care about Transform for two reasons. First, it is the natural place to test whether you understand that stream processing is chunk-by-chunk, not record-by-record — the "a line can span two chunks" problem lives here. Second, writing a correct Transform (buffering partial data, flushing the tail, propagating errors) demonstrates real stream fluency in a way that piping two file streams together does not.

The concept, simply

A **Duplex** stream is both Readable and Writable in one object, but the two sides are *independent* — what you write has no defined relationship to what you read. A TCP socket is the canonical example: you write bytes out to the peer and read bytes in from the peer, and those are two separate conversations that just happen to share a connection object.

A **Transform** is a special Duplex where the two sides *are* related: the readable output is computed from the writable input. You write data in, the stream transforms each chunk, and the result comes out the readable side. Gzip is a Transform: bytes in, smaller bytes out. A cipher is a Transform: plaintext in, ciphertext out. Because a Transform is a

Readable, you can pipe from it; because it is a Writable, you can pipe into it — which is exactly what lets you chain many of them into a pipeline.



A Transform reads a chunk, transforms it, and writes the result to the next stage; chaining Transforms builds a constant-memory processing pipeline.

The mental model: a Transform sits in the *middle* of a pipe. Data flows in one end, gets reshaped, flows out the other. Because it participates in the same backpressure protocol as any stream, a slow stage automatically throttles the fast stages before it — the whole chain self-regulates.

How it works

To build a Transform you implement two methods. `_transform(chunk, encoding, callback)` is called for every chunk written to the stream. Inside it you do your work and emit output by calling `this.push(outputChunk)` zero or more times, then call `callback()` to signal you are ready for the next chunk. You can also pass the transformed value straight to the callback: `callback(null, outputChunk)` is shorthand for one push. Crucially, the number of chunks out does not have to match the number in — you might buffer several input chunks and emit one output, or receive one and emit many.

The second method, `_flush(callback)`, is called **once**, after the last chunk has been written and before the stream ends. This is where you emit anything you were holding back — the final line of a file that had no trailing newline, the last block of a cipher, buffered state that only makes sense at the end. Forgetting `_flush` is the classic source of “my parser drops the last record” bugs.

Backpressure through a Transform. A Transform has *two* internal buffers — one on its writable side and one on its readable side, each with its own highWaterMark. When the downstream consumer is slow, the Transform's readable buffer fills, which causes it to stop calling your `_transform` callback promptly, which causes its writable buffer to fill, which makes `write()` return `false` to whatever is feeding it — and so backpressure propagates all the way back up the chain to the original source. You get this for free as long as you honor the callback contract and do not call `push` unboundedly while ignoring its return value.

Object mode matters a lot for Transforms. A byte-mode Transform reshapes Buffers; an object-mode Transform can turn bytes into structured records (parse a CSV row into an object) or objects into bytes (serialize a record to a JSON line). A common pattern is a pipeline where the first Transform is `{ writableObjectMode: false, readableObjectMode: true }` — bytes in, objects out — because the two sides of a Duplex can have different modes.

Code examples

1. Use the built-in Transforms — gzip a file. `zlib` ciphers and compressors are all Transform streams; you rarely need to write compression yourself.

JAVASCRIPT Copy

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';

await pipeline(
  createReadStream('report.csv'),
  createGzip(),                       // Transform: bytes -> compressed by
  createWriteStream('report.csv.gz')
);
console.log('Compressed in constant memory.');
```

2. A byte-mode Transform: uppercase every chunk. The simplest possible custom Transform — note the constructor-function form.

```
import { Transform } from 'node:stream';

const upper = new Transform({
  transform(chunk, _enc, cb) {
    // chunk is a Buffer; reshape and push the result.
    cb(null, chunk.toString('utf8').toUpperCase());
  }
});

process.stdin.pipe(upper).pipe(process.stdout);    // type text, see it upp
```

3. A line splitter — the "chunk boundary" problem solved. This is the pattern interviewers love: buffer partial input, emit complete lines, flush the tail.

JAVASCRIPT Copy

```
import { Transform } from 'node:stream';

class SplitLines extends Transform {
  #tail = '';
  constructor() { super({ readableObjectMode: true }); }  // emit strings
  _transform(chunk, _enc, cb) {
    const lines = (this.#tail + chunk).split('
');
    this.#tail = lines.pop();          // keep the possibly-incomplete las
    for (const line of lines) this.push(line);
    cb();
  }
  _flush(cb) {
    if (this.#tail) this.push(this.#tail);    // emit the final line (no ne
    cb();
  }
}

import { createReadStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
await pipeline(
  createReadStream('access.log'),
  new SplitLines(),
```

```
      if (line.includes(' 500 ')) this.push(line + '
');
      cb();
    }
  }),
  process.stdout
);
```

4. A full processing pipeline: parse → filter → serialize. Object-mode Transforms chained end to end, in constant memory over any file size.

JAVASCRIPT Copy

```
import { Transform } from 'node:stream';
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';

const parseCsvRow = new Transform({
  readableObjectMode: true,
  transform(buf, _e, cb) {
    for (const line of buf.toString().split('
')) {
      if (!line.trim()) continue;
      const [id, name, amount] = line.split(',');
      this.push({ id, name, amount: Number(amount) });
    }
    cb();
  }
});

const keepBigSpenders = new Transform({
  objectMode: true,
  transform(row, _e, cb) {
    if (row.amount ≥ 1000) this.push(row);
    cb();
  }
});

const toJsonl = new Transform({
```

```
});

await pipeline(
  createReadStream('transactions.csv'),
  parseCsvRow,
  keepBigSpenders,
  toJsonl,
  createWriteStream('big-spenders.jsonl')
);
```

Common mistakes & gotchas

- **Assuming one input chunk equals one logical record.** A chunk can split a line, a CSV row, or even a multi-byte character. You must buffer partial data across `_transform` calls and reassemble it — that is what the `#tail` field in the splitter does.
- **Forgetting `_flush`.** Anything you were holding at the end — the last line, a final cipher block — is silently lost if you don't emit it in `_flush`. This is the number-one Transform bug.
- **Not calling the `_transform` callback** (or calling it twice). One call per invocation: the stream stalls forever if you skip it, and throws if you call it twice.
- **Doing heavy synchronous CPU work inside `_transform`.** It runs on the event loop; a slow synchronous transform blocks everything. For CPU-bound transforms, offload to a worker thread.
- **Mismatched object modes.** Piping an object-mode Readable into a byte-mode Writable (or vice versa) throws or produces `[object Object]` garbage. Set `readableObjectMode` / `writableObjectMode` deliberately.
- **Treating a Duplex like a Transform.** A plain Duplex's read and write sides are independent; do not assume writing to it produces related output on its readable side.

Best practices

custom logic.

- **Always implement** `_flush` if your Transform buffers any state, and test the "last record has no trailing delimiter" case explicitly.
- **Keep** `_transform` **non-blocking.** If work is CPU-heavy, push it to a `worker_threads` pool; if it is async I/O, call the callback when the async work completes so backpressure is respected.
- **Choose object modes explicitly per side** using `readableObjectMode` and `writableObjectMode` for parsers (bytes in, objects out) and serializers (objects in, bytes out).
- **Compose small, single-purpose Transforms** (parse, filter, map, serialize) and wire them with `pipeline()` rather than building one giant do-everything stream.

⭐ Interview lens

Q: What is the difference between a Duplex and a Transform stream?
Both are readable and writable. In a Duplex the two sides are independent (e.g., a TCP socket — incoming and outgoing data are unrelated). A Transform is a Duplex where the readable output is derived from the writable input (e.g., gzip, a cipher, a parser). Transform is effectively Duplex plus the constraint "out = f(in)."

Q: What are `_transform` **and** `_flush` **for?**
`_transform(chunk, enc, cb)` runs per input chunk: do the work, `push()` output (or pass it to `cb`), then call `cb` to accept the next chunk. `_flush(cb)` runs once at the very end, before `'end'`, and is where you emit any buffered leftover — the final partial line, the last cipher block.

Q: You write a newline-delimited parser and it always loses the last line. Why?
(Classic trap.)
Two likely causes: you only emit a line when you see a , so a final line without a

a chunk boundary or arrive in the very last chunk.

Q: How does backpressure work through a Transform stream?

A Transform has two internal buffers, one per side. When downstream is slow, its readable buffer fills, so it delays calling your `_transform` callback, so its writable buffer fills, so `write()` to it returns `false` — and the pressure propagates back up the whole chain. It works automatically as long as you honor the callback contract.

Q: Your CPU-bound Transform makes the whole server unresponsive under load. Why, and how do you fix it?

`_transform` runs on the single event-loop thread; a heavy synchronous computation blocks all other requests. Move the CPU work to a `worker_threads` pool and have `_transform` await the worker's result before calling its callback, so the loop stays free.

Q: Can the number of output chunks differ from the number of input chunks in a Transform?

Yes. You can call `push` zero, one, or many times per `_transform` call, and you can buffer several input chunks and emit a single output later (or the reverse). A line splitter and a compressor both change the chunk count.

Cheat-card

- **Duplex** = readable + writable, sides *independent* (TCP socket).
- **Transform** = Duplex where *out = f(in)* (gzip, cipher, parser). Pipe into it *and* out of it.
- `_transform(chunk, enc, cb)`: do work, `push()` or `cb(null, out)`, then call `cb`. Chunks in ≠ chunks out.

- **Chunk ≠ record**: buffer partial data across calls; reassemble logical units yourself.
- **Modes**: set `readableObjectMode` / `writableObjectMode` per side. CPU-heavy work → worker thread.

QUICK CHECK — no marks, just for you.

1. What distinguishes a Transform stream from a plain Duplex stream?

- ◯ A Transform is read-only while a Duplex is write-only
- ◯ In a Transform the readable output is computed from the writable input; in a Duplex the two sides are independent
- ◯ A Transform cannot be piped into, only piped from
- ◯ A Duplex can change chunk contents but a Transform cannot

Check answer

2. A newline-delimited parser built as a Transform always loses the last line. What is the fix?

- ◯ Increase the highWaterMark so the last line fits in one chunk
- ◯ Call push(null) at the start of _transform to force a final flush
- ◯ Emit the buffered tail in _flush, which runs once at the end, and buffer partial content across _transform calls
- ◯ Set objectMode to false so lines are not dropped

Check answer

3. Can output chunks differ from the number of input chunks in a Transform?

- No; Transform streams must emit exactly one output chunk per input chunk
- Only if objectMode is enabled on both sides
- No; the chunk counts must match or backpressure breaks
- Yes; you can push zero, one, or many times per _transform call, and you can buffer several inputs and emit one output (or the reverse)

Check answer

‹ Previous
Readable & Writable Streams

Next ›
Backpressure, pipe & pipeline

LEARN MORE

ARTICLES & DOCS

- Stream | Node.js Documentation
  nodejs.org
- Node.js Streams: Everything you need to know
  freecodecamp.org

VIDEOS



Learn Node.js Streams in 25 minutes | NodeJS Tutorials for Beginners
Dipesh Malvia

Hand-picked references for this lesson.